SeisTracer Package Manual

Caio Ciardelli
Northwestern University
Under the supervision of Prof. Suzan van der Lee

December 19, 2025

Contents

1 Introduction

The SeisTracer Package is a geosciences software package developed for computing seismic ray paths and travel times in 1D spherical Earth models. It employs dense precomputed tables, the Newton-Raphson method, and analytical expressions for distances and travel times, without relying on Earth-flattening transformations. The package supports a wide range of seismic phases, including teleseismic, core, diffracted, and higher-order phases, with options for amplitude computations and visualizations. Written in Python and C, it is designed for efficient forward modeling of seismic wave propagation. Key features:

- Load and plot 1D velocity models (Vp, Vs, density vs. depth).
- Precompute tables of ray parameters and distances for models and depths.
- Compute travel times, ray paths, take-off angles, and amplitudes for specified phases or all available.
- Support for minor/major arcs, multiple loops, and diffracted phases (e.g., Pdiff, Sdiff).
- Visualize ray paths with branch colors and discontinuity circles.
- Generate and plot travel-time curves.
- Compute and plot amplitude coefficients at discontinuities.

This manual provides a comprehensive guide to installing, compiling, running, and understanding the package. It includes details on the directory structure, dependencies, key scripts and programs, input/output files, and usage examples. The program is licensed under the GNU General Public License (GPL) version 3 or later. See the LICENSE file for details.

2 Dependencies

The SeisTracer Package requires the following software and libraries:

- **Bash**: For running shell scripts (e.g., create_tables.bash).
- **GCC**: C compiler for building the C binaries and shared libraries (tables.c, tracer.c, etc.).
- **Python 3.8 or later**: For the main Python module (seistracer.py).
- **Python libraries**:
  - NumPy: Data processing and arrays.
  - SciPy: Interpolation (interp1d).
  - Matplotlib: Plotting ray paths and models.
  - ctypes: Interfacing with C shared libraries.
  - warnings, os, sys, copy, math, struct, glob, pathlib: Standard libraries for utilities.

- **Make**: For compiling via Makefile.

Ensure all dependencies are installed on your system. For Python libraries, use pip:

```
pip install numpy scipy matplotlib
```

The package assumes access to predefined models in models/ directory.

3 Installation and Compilation

Clone the repository and navigate to the root directory.

3.1 Compilation

The package uses a Makefile for compilation. To build the C shared libraries (libcoefficients.so, libio.so, libtracer.so) and binaries (tables):

```
make all
```

This compiles the source files in src/ (using headers from src/headers and shared code from src/shared) and places the libraries in lib/ and executables in bin/. To clean previous build artifacts:

```
make clean
```

Note: Ensure GCC is in your PATH. The Makefile assumes standard includes; modify it if needed for custom library paths.

3.2 Setup Python Path

After compilation, add the package directory to PYTHONPATH to import the module:

```
export PYTHONPATH="$HOME/git/seistracer"
```

Add this to your ~/.bashrc or ~/.zshrc for persistence.

3.3 Build Precomputed Tables

After compilation, build tables for all provided models:

```
./create_tables.bash ak135f iasp91 prem rem1d stw105
```

This process generates tables in the directory tables/MODEL/ for depths ranging from 0 to 700 km in 1 km increments for each specified model. The tables for a single model require approximately 9 GB of disk space and may take several hours to compute, depending on the machine's performance. However, they need to be generated only once, after which SeisTracer can efficiently compute a large number of ray paths.

4 Directory Structure

The package is organized as follows:

```
seistracer
├── create_tables.bash
├── docs
│   └── example.py
├── __init__.py
├── LICENSE
├── Makefile
├── models
│   ├── ak135f.model
│   ├── iasp91.model
│   ├── prem.model
│   ├── rem1d.model
│   └── stw105.model
├── seistracer.py
├── README.md
├── setup
│   └── constants.h
├── src
│   ├── headers
│   │   ├── amplitudes.h
│   │   ├── coefficients.h
│   │   ├── constants_export.h
│   │   ├── enums.h
│   │   ├── exmath.h
│   │   ├── heapsort.h
│   │   ├── io.h
│   │   ├── permutations.h
│   │   ├── ray.h
│   │   ├── source_and_model.h
│   │   └── structs.h
│   ├── shared
│   │   ├── amplitudes.c
│   │   ├── coefficients.c
│   │   ├── heapsort.c
│   │   ├── io.c
│   │   ├── permutations.c
│   │   ├── ray.c
│   │   └── source_and_model.c
│   ├── tables.c
│   └── tracer.c
└── tables
    ├── ak135f
    ├── iasp91
    ├── prem
    └── rem1d
```

```
└── stw105
```

Key directories:

- **docs**: Contains example usage scripts (example.py).
- **models**: Predefined 1D Earth models (e.g., iasp91.model) in text format.
- **setup**: Configuration header (constants.h) for C programs.
- **src**: C source code, including main programs (tables.c, tracer.c), headers, and shared utilities for amplitudes, ray tracing, etc.
- **tables**: Precomputed tables for models (e.g., codes_0.list, phases_0.list, table_0.bin per depth and model).

5 Running the Program

After building tables, use the Python module for computations. Import seistracer and create a Tracer object with a model. The module handles phase calculations, visualizations, and more. Key usage examples are provided in Section 7 and docs/example.py.

6 Key Components and Usage

6.1 create_tables.bash

This Bash script builds precomputed tables for specified models across depths 0 to 700 km using the tables binary. Usage:

```
./create_tables.bash MODEL1 [MODEL2 ...]
```

Example:

```
./create_tables.bash ak135f iasp91 prem rem1d stw105
```

Output: In tables/MODEL/: codes_DEPTH.list (permutation codes), phases_DEPTH.list (phase names and counts), table_DEPTH.bin (binary blocks with np, index, delta, psph) for each depth. Notes: Runs in parallel for depth ranges; progress messages.

6.2 seistracer.py

Core Python module; loads models/tables via ctypes, computes paths/times/amplitudes. Classes:

- Constants: Exports C constants (e.g., MAX_PHASES).
- Phases: Manages phase data; methods: __str__ (list phases), getInfo (print info), getDict (dict of ttime/take-off/etc.), getPath (ray paths), plotPath (visualize paths).
- Tracer: Main class; loads model, plots model/coefficients, computes coefficients, ttimesAndPaths (returns Phases), getTravelTimes (dict), plotTravelTimes (curves).

6.3 tables.c (bin/tables)

Generates tables; precomputes coefficients, permutations, sorts by delta. Usage: ./bin/tables DEPTH MODEL (called by create_tables.bash). Output: As above. Notes: Uses Newton-Raphson for solutions; analytical integrals.

6.4 tracer.c (lib/libtracer.so)

Computes ttimes/paths from tables; uses heapsort for ordering. Functions: ttimesAndPaths (core; computes times/paths). Notes: Adjusts delta; interpolates psph; computes/rescales paths.

7 Examples

The docs/example.py script demonstrates various features of the SeisTracer package. Below, we provide the full code with detailed explanations preceding each key section, describing what the code does and why it's useful.

Import the SeisTracer module. This makes all classes and functions available for use, such as Tracer for model loading and phase computations.

```
import seistracer
```

Create a Tracer object with the 'iasp91' model. The Tracer class handles model loading from precomputed tables and provides methods for computations and visualizations. Here, it loads the IASP91 1D Earth model.

```
tracer = seistracer.Tracer(model='iasp91')
```

Plot the loaded velocity model. This generates a figure showing P-wave velocity (Vp), S-wave velocity (Vs), and density (rho) as functions of depth, with labeled discontinuities (e.g., Moho, 410 km, 660 km, CMB, ICB). Useful for verifying the model structure.

```
tracer.plotModel()
```

Compute travel times and ray paths for phases 'P' and 'S' at source depth 0 km and epicentral distance (delta) 20 °. The ttimesAndPaths method returns a Phases object containing computed arrivals. Specifying phases limits computations to those; default is 'all'.

```
phases = tracer.ttimesAndPaths(source_depth=0, delta=20, phases=['P', '
    S'])
```

Print detailed information for all computed phases. The getInfo method displays a formatted table with phase name, travel time (ttime), take-off angle, dT/dD (ray parameter), and d2T/dD2 (second derivative for spreading).

```
phases.getInfo()
```

Plot the ray paths for all computed phases. The plotPath method visualizes paths in a circular Earth section, with colored branches (e.g., P red, S blue, K orange) and black discontinuity circles. Source is yellow star, receiver red triangle.

```
phases.plotPath()
```

Plot travel-time curves for default phases ('P' and 'S'). The plotTravelTimes method generates curves of travel time (minutes) vs. delta (degrees), useful for understanding phase arrivals over distances.

```
tracer.plotTravelTimes()
```

Compute diffracted phases 'Pdiff' and 'Sdiff' at delta 150 °. Demonstrates handling of phases beyond the direct wave cutoff, extending along the CMB.

```
phases = tracer.ttimesAndPaths(source_depth=0, delta=150, phases=['
    Pdiff', 'Sdiff'])
```

Plot travel-time curves including diffracted phases. Specifies phases to include both direct and diffracted for comparison.

```
tracer.plotTravelTimes(phases=['P', 'S', 'Pdiff', 'Sdiff'])
```

Plot curves for a custom list of multiple phases, showcasing reflected, core, and converted phases.

```
tracer.plotTravelTimes(phases=['PP', 'SS', 'PS', 'PKP', 'SKP', 'SKKS',
                               'PKIKP', 'PKIKS', 'PKJKP', 'SKS660t+P'])
```

Plot amplitude coefficients at all discontinuities. Visualizes amplitude and energy flux vs. incidence angle for reflections/transmissions at each interface (e.g., surface, Moho).

```
tracer.plotAmplitudeCoefficients(discontinuities='all')
```

Compute specific phases 'P', 'P410-P', and 'SKS660t+P' at delta 98 °, demonstrating phase naming for reflections and conversions.

```
phases = tracer.ttimesAndPaths(source_depth=0, delta=98,
                               phases=['P', 'P410-P', 'SKS660t+P'])
```

Compute all minor-arc phases at depth 480 km and delta 140 °.

```
phases = tracer.ttimesAndPaths(source_depth=480, delta=140, arc='minor'
    )
```

Compute all major-arc phases (counter-clockwise propagation).

```
phases = tracer.ttimesAndPaths(source_depth=480, delta=140, arc='major')
```

Compute both minor and major arcs with amplitude computations enabled.

```
phases = tracer.ttimesAndPaths(source_depth=480, delta=140, arc='both',
                               compute_amplitudes=True)
```

Print info for selected phases from the computed set.

```
phases.getInfo(['pPKJKS', 'P410-ScSScS'])
```

Retrieve ray path data (radii, deltas, branches) for a single phase as a dictionary, useful for further analysis or export.

```
paths = phases.getPath(['pPKJKS'])
```

Retrieve all phase information (ttime, take-off, etc.) for a phase as a dictionary.

```
phases_dict = phases.getDict(['pPKJKS'])
```

Plot paths for a list of specific complex phases.

```
phases.plotPath(['PKIIKP', 'PKP660t+SScP', 'PKJKS',
                 'SPS660-S', 'ScSScS660-ScS_M', 'pPKJKS',
                 'pPmt-ScSScS', 'P410-ScSScS', 'sSKIKPPm+P'])
```

Compute minor-arc phases including higher-order multiples up to 2 loops (e.g., for long-distance multiples).

```
phases = tracer.ttimesAndPaths(source_depth=480, delta=140, arc='minor',
                               min_loop=0, max_loop=2)
```

Compute all available phases at depth 20 km and delta 150 °.

```
phases = tracer.ttimesAndPaths(source_depth=20, delta=150)
```

For contributions or issues, contact the author.